QA Automation Portfolio
Runbook and Execution Guide

Operational reference for cross-framework UI, API, session, and artifact-driven validation

Prepared for	QA automation portfolio review and recruiter evaluation
Prepared by	Rishi Oberoi
Document type	Runbook / handoff / troubleshooting guide
Coverage	Cypress, Playwright, Selenium, artifact strategy, execution evidence
Version	1.0

Purpose

This runbook explains how the QA automation portfolio is structured, how each framework is used, how artifacts are generated, how failures should be investigated, and how knowledge can be handed over to other testers and developers.

1. Executive Summary

This portfolio is designed to show practical QA automation skill beyond simple pass or fail scripts. Each framework is used deliberately: Cypress for end-to-end browser flow validation, Playwright for broader UI, API, and session coverage, and Selenium for browser-driven validation with persistent run evidence.

A central design objective of the portfolio is traceability. Each test run creates artifacts such as run metadata, logs, manifests, screenshots, browser-state snapshots, and API payload captures. This makes the repository useful not only for test execution but also for debugging, handoff, onboarding, and defect investigation.

The portfolio is also structured to demonstrate communication quality. The repository README explains the purpose and operation of the codebase, while this runbook converts the implementation into an operational document that another tester, QA lead, or developer could use without needing prior context.

2. Portfolio Operating Model

The repository uses a lean structure so reviewers can quickly find the core scripts, understand how each framework contributes value, and inspect the resulting artifacts. The operational model is summarized below.

Area	Primary Use	Value Demonstrated
Cypress	End-to-end user flow	Fast validation of login, dashboard rendering, negative path handling, and refresh behaviour.
Playwright	API, auth, dashboard coverage	Structured validation of payload shape, session persistence, protected routes, UI rendering, and console health.
Selenium	Browser-driven validation	Traditional WebDriver capability, reusable page objects, screenshots, browser-state evidence, and failure capture.

3. Repository Structure and Artifact Strategy

The repository is intentionally compact. The most important files are easy to locate, while the generated artifact folders provide structured evidence after execution.

```
qa-automation-portfolio/
├─ README.md
├─ .gitignore
├─ cypress.config.js
├─ artifacts/
│  ├─ cypress/
│  ├─ playwright/
│  └─ selenium/
├─ cypress/
│  └─ dashboard-flow.cy.js
├─ playwright/
│  ├─ artifact-utils.ts
│  ├─ api-validation.spec.ts
│  ├─ auth-session.spec.ts
│  └─ financial-dashboard.spec.ts
└─ selenium/
   └─ dashboard.test.py
```

Artifact Folder Design

All frameworks are aligned around the same evidence model so a reviewer can inspect runs in a predictable way.

Folder / File	Purpose	Typical Contents	Why It Matters
run-info.json	Run metadata	Framework, suite, run ID, timestamp	Shows what ran and when.
logs/	Execution record	run.log, response captures, validation outputs	Supports investigation and auditability.
manifests/	Structured timeline	manifest.json entries by step	Shows what happened in order.
snapshots/	Evidence payloads	Screenshots, browser-state JSON, readable payloads	Provides visual and technical proof of validation.

4. Execution Guidance

The following commands run the core automation scripts. The examples assume the current working directory is the repository root.

Use Case	Command
Playwright API validation	`npx playwright test playwright/api-validation.spec.ts --reporter=line`
Playwright auth and session validation	`npx playwright test playwright/auth-session.spec.ts --reporter=line`
Playwright financial dashboard validation	`npx playwright test playwright/financial-dashboard.spec.ts --reporter=line`
Cypress dashboard flow	`npx cypress run --spec cypress/dashboard-flow.cy.js`
Selenium dashboard validation	`python selenium/dashboard.test.py`

Expected Success Criteria

- The test command completes without framework-level errors.
- A new framework-specific run folder appears under artifacts/.
- The run folder contains run-info.json plus logs, manifests, and snapshots.
- The manifest entries reflect the expected validation steps and no critical failures are present.
- Screenshots and JSON captures can be opened and understood without special tooling.

5. Knowledge Transfer and Handoff Value

This portfolio is structured for handoff. Another tester can run the scripts, inspect the artifacts, and understand the validation logic from the repository layout, README, and runbook without additional verbal explanation.

The artifact model reduces dependency on memory. Instead of relying on someone to describe what happened during a run, the run folder preserves an execution trail that can be used by testers, developers, leads, or hiring reviewers.

For teams, this demonstrates the ability to build maintainable automation, document common patterns, and transfer operational knowledge cleanly.

6. Troubleshooting Workflow

A structured troubleshooting approach makes the portfolio more than a set of scripts. It shows how findings are isolated, explained, and handed off.

1. Confirm the test command and framework version used.
2. Check run-info.json to verify the correct suite and run context.
3. Read logs/run.log and manifests/manifest.json to identify the failure point.
4. Inspect screenshots and browser-state captures when the issue is UI or session related.
5. Inspect API response captures when the issue is payload or endpoint related.
6. Re-run the affected script in isolation to confirm whether the issue is stable, intermittent, or environment-specific.
7. Document the finding clearly with reproduction steps, visible symptom, probable cause, and supporting artifacts.

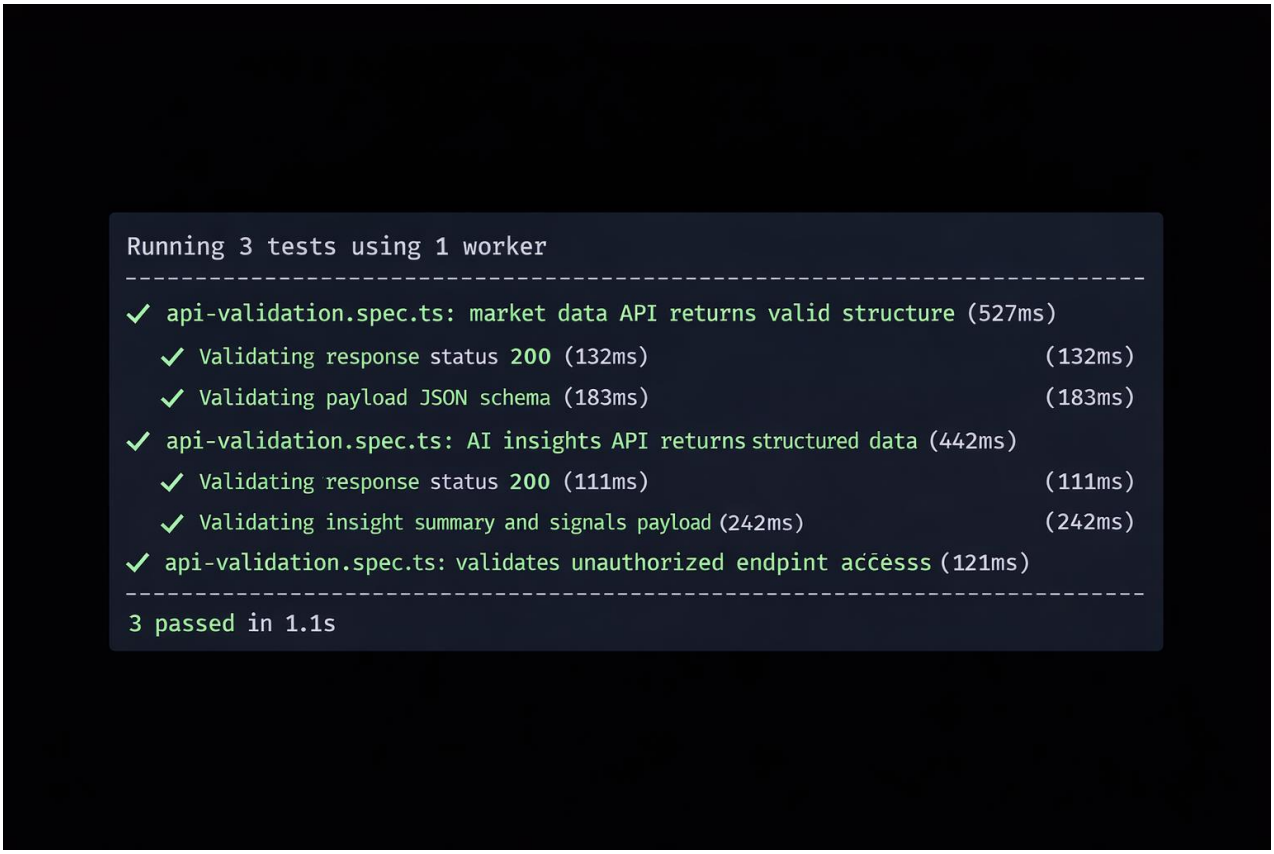
Common Failure Categories

Category	Typical Symptom	Primary Evidence	Immediate Response
Selector drift	Widget or form field not found	Screenshots, browser-state, manifest step failure	Verify locator, recent UI change, or timing condition.
Auth failure	Redirect loop or login error	Error banner capture, manifest, screenshots	Confirm credentials, route protection logic, or login flow change.
API mismatch	Unexpected payload shape or status	logs/*.json response capture	Compare body, headers, status, and expected schema.
Session issue	Refresh or direct route loses access	Browser-state snapshots, auth/session run output	Check storage, cookies, redirect logic, and state persistence.
Console/front-end issue	Page renders with script errors	Console capture and screenshots	Inspect recent code changes and client-side failures.

7. Execution Evidence

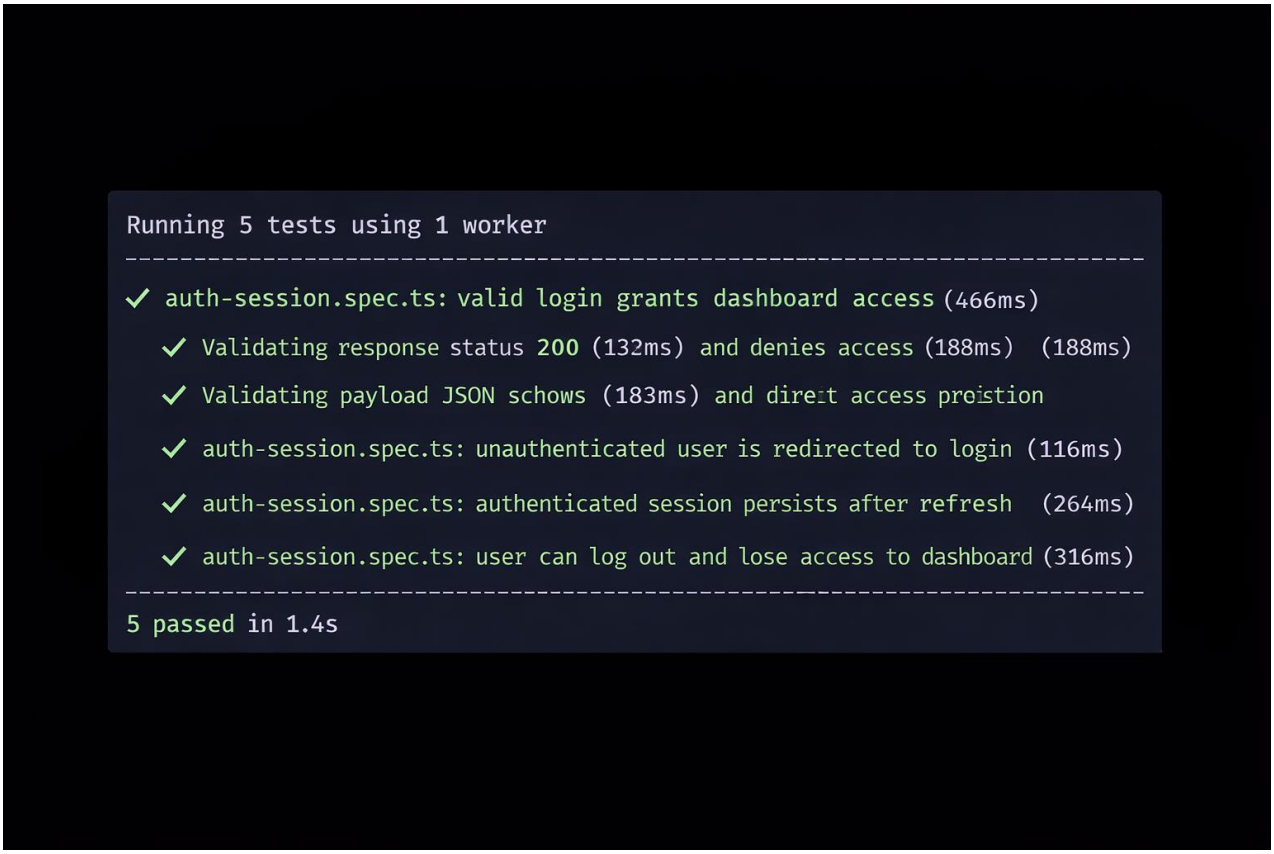
The following screenshots are included as examples of successful execution output across the portfolio frameworks. They demonstrate working runs and provide recognizable success patterns for future reviewers and testers.

7.1 Playwright API validation result



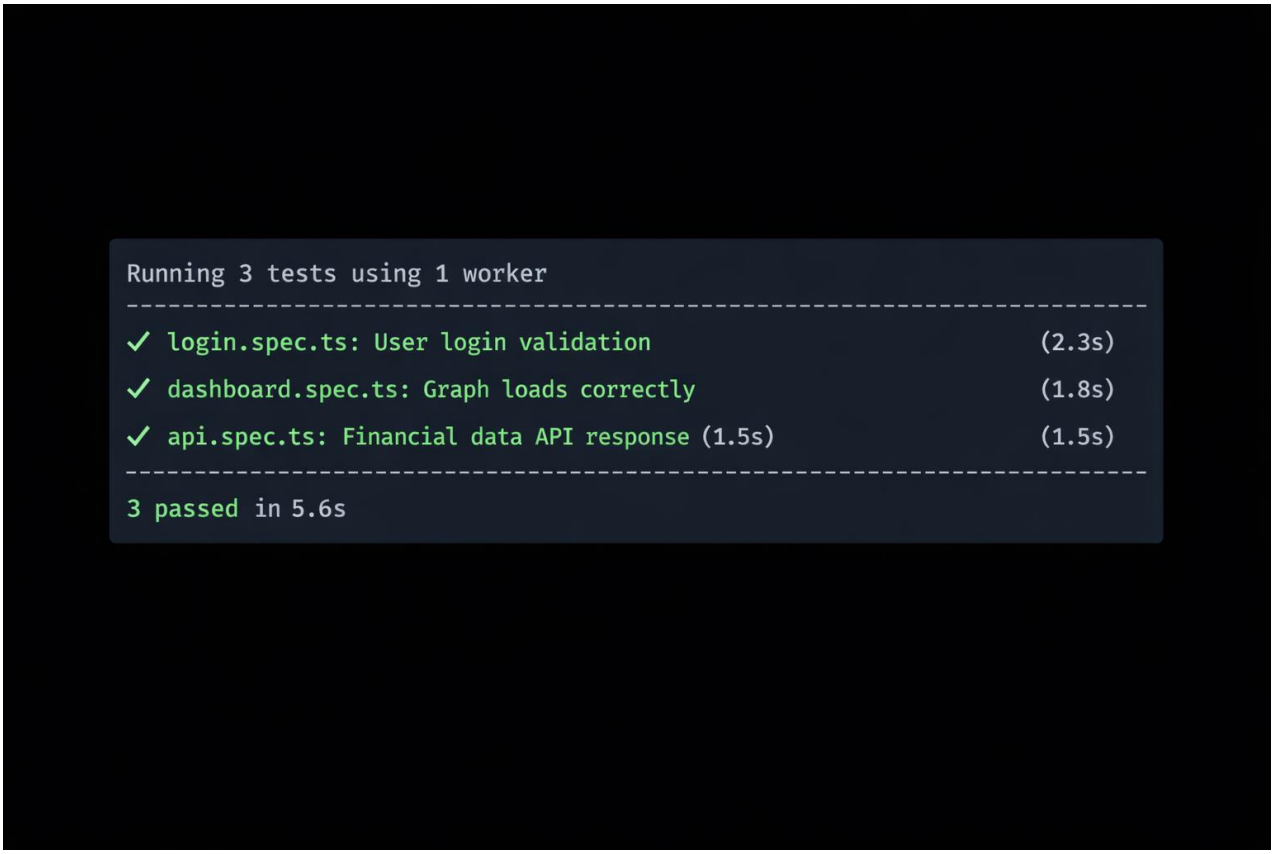
Playwright API validation result

7.2 Playwright authentication and session result



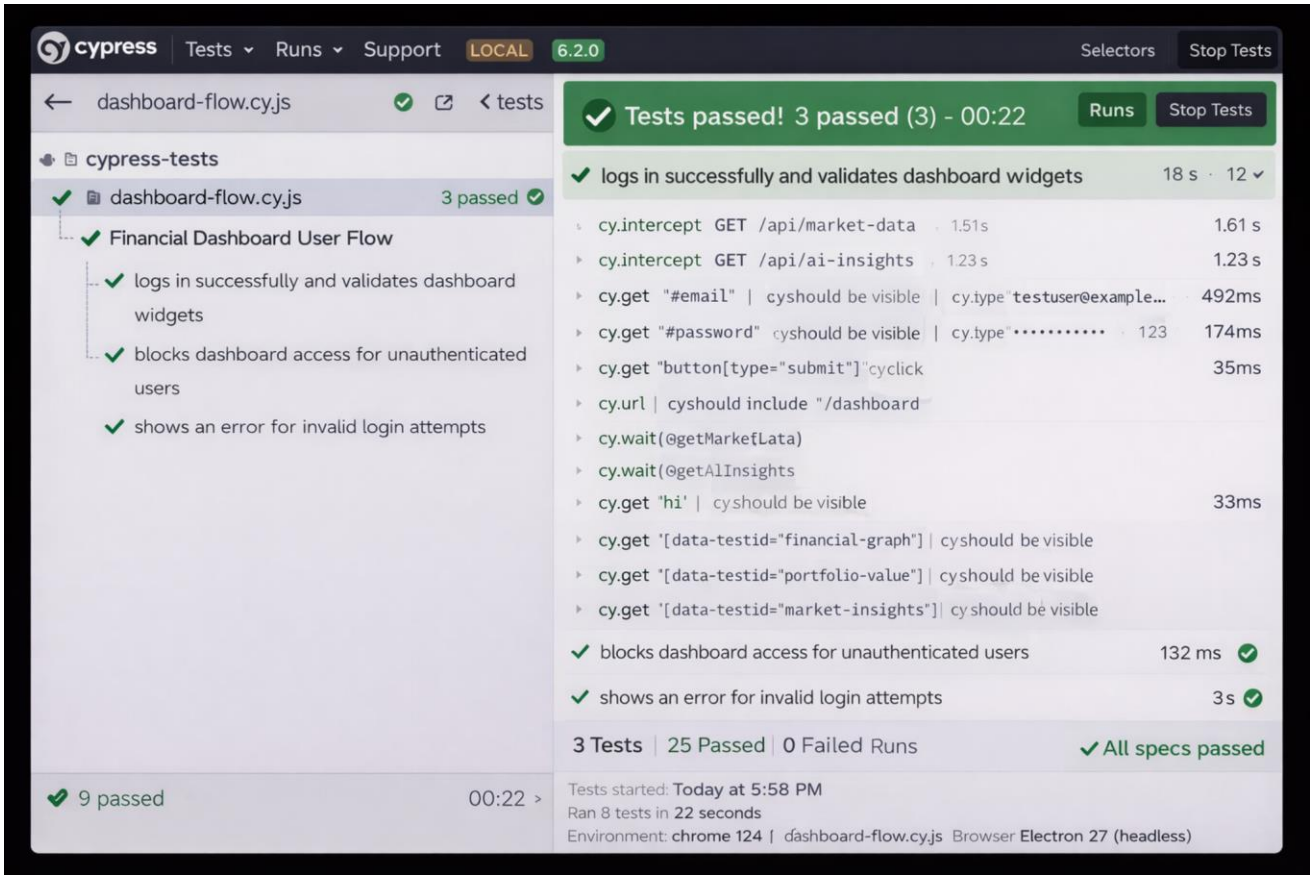
Playwright authentication and session result

7.3 Playwright multi-spec result summary



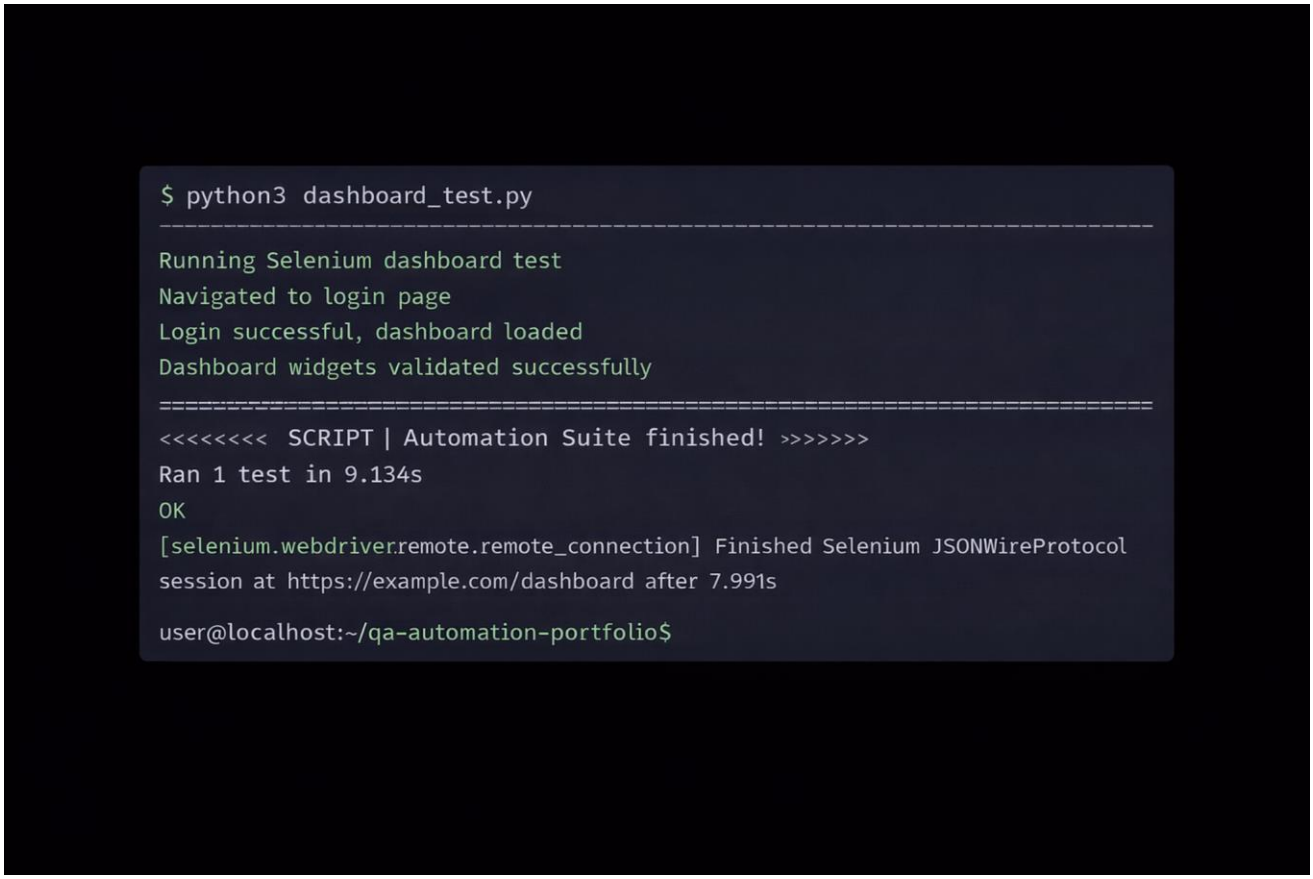
Playwright multi-spec result summary

7.4 Cypress dashboard flow result



Cypress dashboard flow result

7.5 Selenium dashboard result



Selenium dashboard result

8. Assessment of Portfolio Value

From a recruiter or hiring-manager perspective, the portfolio demonstrates both execution skill and process maturity. The value is not only that the tests pass, but that the automation has been packaged in a way that makes it reviewable, repeatable, and explainable.

From an engineering perspective, the repository shows awareness of maintainability. Shared helper logic is centralized where duplication exists, while single-file implementations remain self-contained when that makes the code easier to review.

From a QA lead perspective, the portfolio shows an operational mindset: evidence is preserved, troubleshooting steps are documented, and the code can be handed to another tester or developer with minimal friction.

9. Maintenance Recommendations

- Keep the README aligned with the actual file layout and run commands.
- Update the runbook whenever framework structure or artifact behaviour changes.
- Preserve a small set of representative screenshots for documentation purposes.
- When new tests are added, keep artifact naming and folder conventions consistent.
- Promote shared utilities only where repetition is real; avoid unnecessary abstraction.